incorporating it into your pages and controllers themselves, you're able to reuse the business logic of your application with multiple UI paradigms.

TIP Where possible, keep your page handlers and controllers as simple as practicable. Move all your business logic decisions into the services that make up your application model, and keep your Razor Pages and API controllers focused on the mechanics of interacting with a user or client.

After the application model has returned the data necessary to service the request—the fruit objects in the `apples` category—you see the first significant difference between API controllers and Razor Pages. Instead of adding values to the `PageModel` to be used in a Razor view, the action method creates an *API model*. This is analogous to the `PageModel`, but rather than containing data used to generate an HTML view, it contains the data that will be sent back in the response.

DEFINITION *View models* and `PageModels` contain both the *data* required to build a response and *metadata* about *how* to build the response. API models typically contain only the data to be returned in the response.

When we looked at the Razor Pages app, we used the `PageModel` in conjunction with a Razor view template to build the final response. With the web API app, we use the API model in conjunction with an output formatter. An *output formatter*, as the name suggests, serializes the API model into a machine-readable response, such as JSON or XML. The output formatter forms the V in the web API version of MVC by choosing an appropriate representation of the data to return.

Finally, as for the Razor Pages app, the generated response is sent back through the middleware pipeline, passing through each of the configured middleware components, and back to the original caller.